PYTHON ACTIVITY BASED LEARNING

PROBLEM-4:

Student Marks Matrix Analysis: Analyse a matrix of student marks across subjects, calculate row-wise and column-wise statistics, identify top performers, and detect subjects with low average performance.

Objective:

To develop a Python program that analyses a matrix of student marks across multiple subjects, calculates row-wise and column-wise statistics, identifies top-performing students, and detects subjects with low average marks to evaluate student performance and identify areas that need improvement.

Algorithm:

1. Start the program.
2. Input the number of students (\(n\)) and subjects (\(m\)).
3. Input the names of all students and subjects.
4. Input the marks of each student in every subject and store them in a matrix.
5. Input the low-average threshold.
6. Calculate the total marks and average marks of each student (row-wise analysis).
7. Identify and display the student with the highest average as the top performer.
8. Calculate the average marks of each subject (column-wise analysis).
9. Compare each subject average with the low-average threshold.
10. Store and display the subjects whose averages are below the threshold.
11. If no subject has a low average, display an appropriate message.
12. Stop the program.

Pseudocode:

```
BEGIN

INPUT n, m

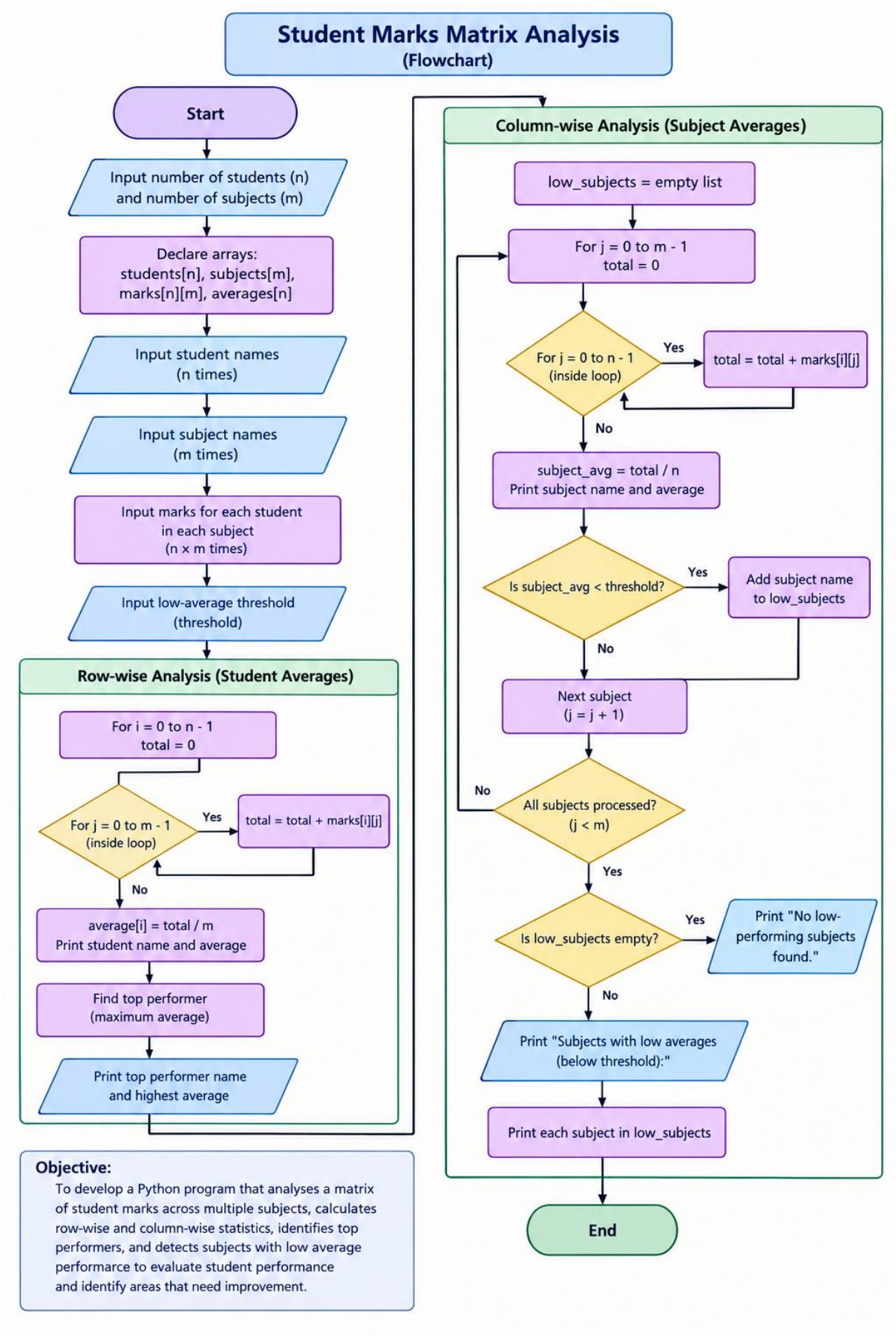
FOR i = 1 TO n
    INPUT student name
END FOR
FOR j = 1 TO m
    INPUT subject name
END FOR
FOR i = 1 TO n
    FOR j = 1 TO m
        INPUT marks[i][j]
    END FOR
END FOR
INPUT low_threshold
// Row-wise analysis
FOR i = 1 TO n
    total = 0
    FOR j = 1 TO m
        total = total + marks[i][j]
    END FOR
    averages[i] = total / m
    DISPLAY student name, averages[i]
END FOR
```

```
// Identify top performer
top_index = 1
FOR i = 2 TO n
    IF averages[i] > averages[top_index] THEN
        top_index = i
    END IF
END FOR
DISPLAY students[top_index]
DISPLAY averages[top_index]
// Column-wise analysis
low_subjects = EMPTY LIST
FOR j = 1 TO m
    total = 0
    FOR i = 1 TO n
        total = total + marks[i][j]
    END FOR
    subject_average = total / n
    DISPLAY subjects[j], subject_average
    IF subject_average < low_threshold THEN
        ADD subjects[j] TO low_subjects
    END IF
END FOR
IF low_subjects IS NOT EMPTY THEN
    DISPLAY low_subjects
ELSE
    DISPLAY "No low-performing subjects found"
```

END IF

STOP

FlowChart:

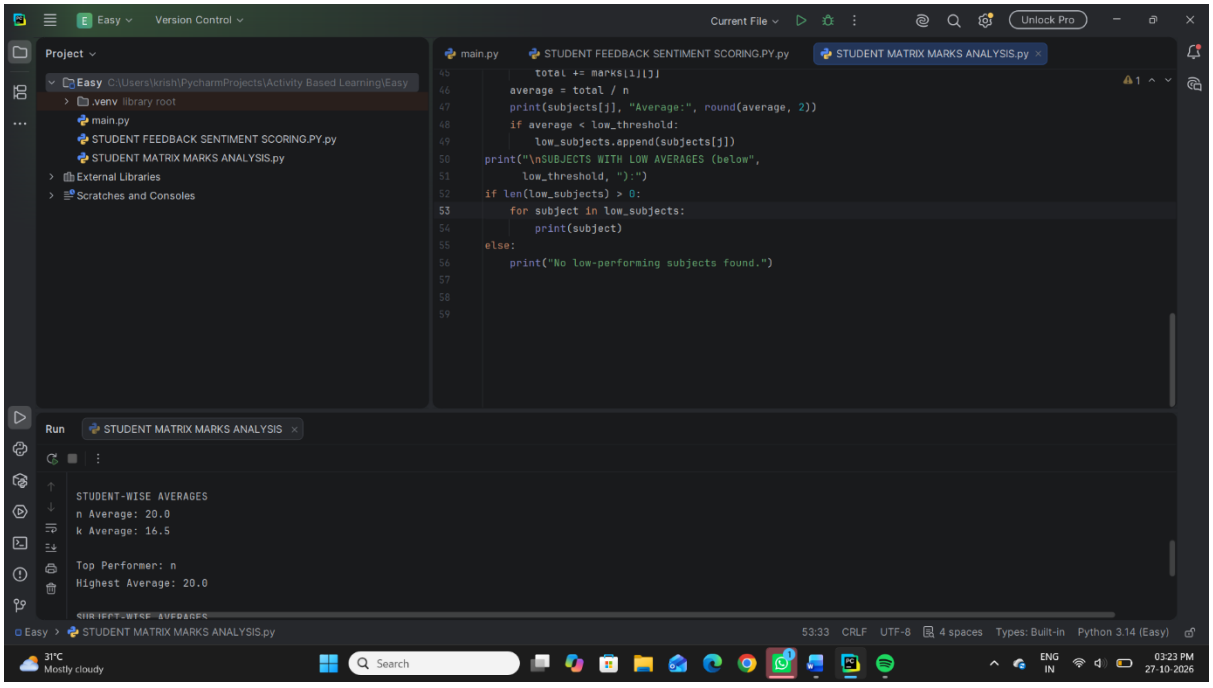

Student Marks Matrix Analysis
(Flowchart)
Start
Input number of students (n) and number of subjects (m)
Declare arrays: students[n], subjects[m], marks[n][m], averages[n]
Input student names (n times)
Input subject names (m times)
Input marks for each student in each subject (n x m times)
Input low-average threshold (threshold)
Row-wise Analysis (Student Averages)
For i = 0 to n - 1
total = 0
For j = 0 to m - 1 (inside loop)
Yes
total = total + marks[i][j]
No
average[i] = total / m
Print student name and average
Find top performer (maximum average)
Print top performer name and highest average
Column-wise Analysis (Subject Averages)
low_subjects = empty list
For j = 0 to m - 1
total = 0
For j = 0 to n - 1 (inside loop)
Yes
total = total + marks[i][j]
No
subject_avg = total / n
Print subject name and average
Is subject_avg < threshold?
Yes
Add subject name to low_subjects
No
Next subject (j = j + 1)
All subjects processed? (j < m)
No
Yes
Is low_subjects empty?
Yes
Print "No low-performing subjects found."
No
Print "Subjects with low averages (below threshold):"
Print each subject in low_subjects
End
Objective:
To develop a Python program that analyses a matrix of student marks across multiple subjects, calculates row-wise and column-wise statistics, identifies top performers, and detects subjects with low average performarce to evaluate student performance and identify areas that need improvement.

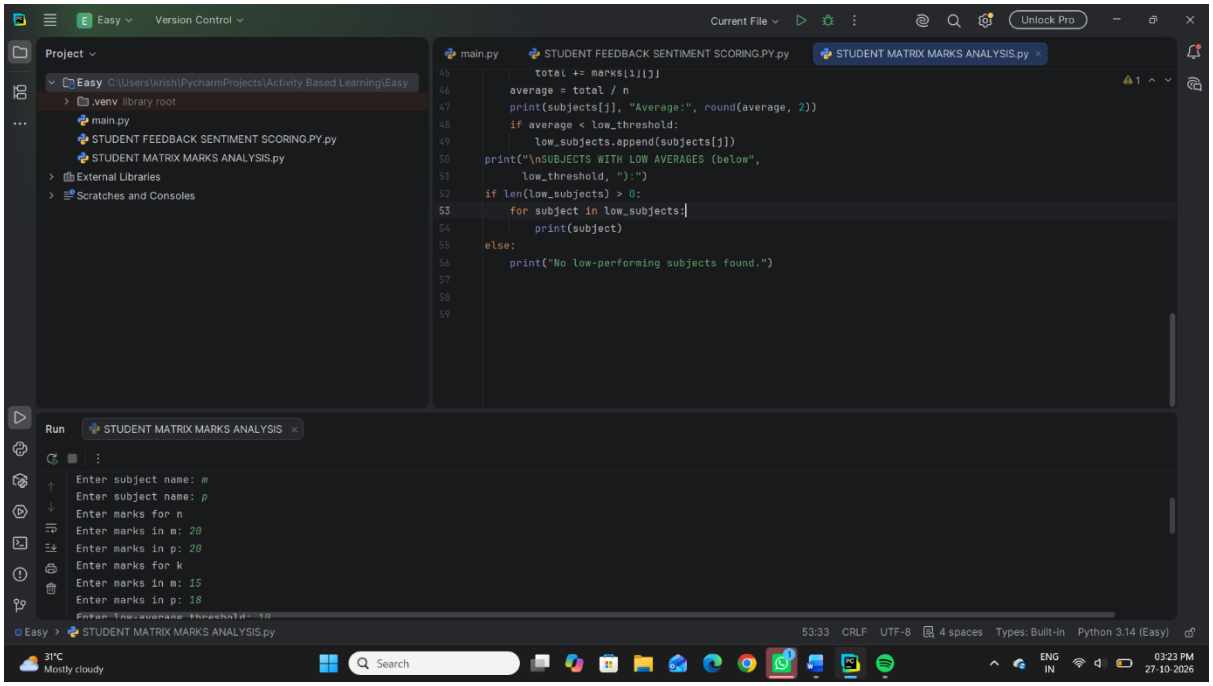
Python Program:

```
# Student Marks Matrix Analysis
n = int(input("Enter number of students: "))
m = int(input("Enter number of subjects: "))
students = []
subjects = []
marks = []
# Input student names
for i in range(n):
    name = input("Enter student name: ")
    students.append(name)
# Input subject names
for j in range(m):
    subject = input("Enter subject name: ")
    subjects.append(subject)
# Input marks matrix
for i in range(n):
    row = []
    print("Enter marks for", students[i])
    for j in range(m):
        mark = float(input("Enter marks in " + subjects[j] + ": "))
        row.append(mark)
    marks.append(row)
low_threshold = float(input("Enter low-average threshold: "))
# Row-wise analysis
averages = []
print("\nSTUDENT-WISE AVERAGES")
for i in range(n):
    total = sum(marks[i])
    average = total / m
    averages.append(average)
    print(students[i], "Average:", round(average, 2))
# Identify top performer
top_index = 0
for i in range(1, len(averages)):
    if averages[i] > averages[top_index]:
        top_index = i
```

```
print("\nTop Performer:", students[top_index])
print("Highest Average:", round(averages[top_index], 2))
# Column-wise analysis
print("\nSUBJECT-WISE AVERAGES")
low_subjects = []
for j in range(m):
    total = 0
    for i in range(n):
        total += marks[i][j]
    average = total / n
    print(subjects[j], "Average:", round(average, 2))
    if average < low_threshold:
        low_subjects.append(subjects[j])
print("\nSUBJECTS WITH LOW AVERAGES (below",
      low_threshold, "):")
if len(low_subjects) > 0:
    for subject in low_subjects:
        print(subject)
else:
    print("No low-performing subjects found.")
```

Output Screen Shots:




Easy
Version Control
Current File
Unlock Pro
Project
Easy C:\Users\krish\PycharmProjects\Activity Based Learning\Easy
.venv library root
main.py
STUDENT FEEDBACK SENTIMENT SCORING.PY.py
STUDENT MATRIX MARKS ANALYSIS.py
External Libraries
Scratches and Consoles
main.py
STUDENT FEEDBACK SENTIMENT SCORING.PY.py
STUDENT MATRIX MARKS ANALYSIS.py
total += marks[i][j]
average = total / n
print(subjects[j], "Average:", round(average, 2))
if average < low_threshold:
low_subjects.append(subjects[j])
print("\nSUBJECTS WITH LOW AVERAGES (below",
low_threshold, "):")
if len(low_subjects) > 0:
for subject in low_subjects:
print(subject)
else:
print("No low-performing subjects found.")
Run
STUDENT MATRIX MARKS ANALYSIS
Enter subject name: m
Enter subject name: p
Enter marks for n
Enter marks in m: 20
Enter marks in p: 20
Enter marks for k
Enter marks in m: 15
Enter marks in p: 18
Easy
STUDENT MATRIX MARKS ANALYSIS.py
53:33
CRLF
UTF-8
4 spaces
Types: Built-in
Python 3.14 (Easy)
31°C
Mostly cloudy
Search
ENG
IN
03:23 PM
27-10-2026